Final Research Report: Integrating Topological Priors and Self-Supervised Learning for Coronary Artery Segmentation

A Chronological Summary of Progress and Experimental Results

1. Clinical Context & Objective

The Clinical Need: Cardiovascular disease is a leading cause of death globally. To properly diagnose, treat, and predict patient cardiac conditions, clinicians must evaluate the degree of coronary artery stenosis (arterial blockage).

The Problem: Pathologists traditionally estimate stenosis visually under a microscope, which introduces inter-observer subjectivity and variations.

The AI Goal: Build an automated deep learning pipeline that segments coronary artery tissue layers—namely the Lumen (flow channel), Intima (plaque layer), and Media (outer wall)—to calculate the stenosis percentage objectively and precisely.

2. Chronological Project Progress

Phase 1: Data Preparation & ROI Extraction
High-resolution Whole-Slide Images (WSIs) were cropped using annotation coordinates exported from QuPath (.geojson files). 16 high-quality Regions of Interest (ROIs) containing coronary arteries were extracted and preprocessed.

Phase 2: Self-Supervised Feature Extraction
Image patches were processed through a pre-trained Vision Transformer (DINOv2) to extract deep, robust visual feature representations of tissues.

Phase 3: Unsupervised Pseudo-Labeling
Applied K-Means Clustering and PCA to group features into 5 distinct classes (Background, Media, Intima, Lumen, Extra) to act as training ground-truth masks.

Phase 4: Baseline Decoder Training
Constructed a 3-layer Convolutional Neural Network (CNN) segmentation decoder on top of the frozen backbone and trained it using Cross-Entropy Loss.

Phase 5: Integrating Topological Constraints (clDice Loss)
Individual pixel classification led to broken artery contours and fragmented lumen regions. We integrated clDice (Centerline Dice) loss into the training loop, enforcing loop connectivity and anatomical correctness.

Phase 6: Performance Engineering (Lazy Loading & Caching)
Implemented lazy loading to load patches on-the-fly, dropping RAM usage from 16,000 MB to under 100 MB. Built an offline crop cache which reduced training speed from 6 hours to under 2 minutes.

Phase 7: Final Training & Validation
Re-trained the model on all 16 slides for 20 epochs, successfully generating smooth, continuous boundaries matching human anatomy.

3. Quantitative Experimental Results

Below is the comparison of the model's performance on the test set before and after implementing the topological constraint:

Metric	Baseline Model	Topology Model (clDice)	Improvement
Average Test Accuracy	57.33%	64.17%	+6.84%
Average Test Mean IoU	0.300	0.381	+0.081

4. Active Code Files Summary

- **scripts/train_baseline.py**: The core training loop containing the Hugging Face model loading, the lazy-loading dataset with offline caching, and the clDice loss.
- **scripts/visualize_predictions.py**: The evaluation script that runs inference and generates the side-by-side comparison overlays.